

CARTRIDGE & CLOUD · DOCUMENTACIÓN RC1

Cartridge & Cloud — Build y Versioning Guide

Edición PDF con identidad visual unificada de VRM Games. El contenido procede íntegramente del archivo Markdown original.

PROYECTO

Cartridge & Cloud

ESTADO

**Fuente vigente de build, versionado,
empaquetado y trazabilidad**

DOCUMENTO

11 / 11_Build_y_Versioning_Guide.md

AUTORÍA

VRM Games / Blas Luis Rocha González

VERSIÓN

1.0

FECHA DOCUMENTAL

2026-07-01

Control y metadatos del documento

Archivo fuente: 11_Build_y_Versioning_Guide.md
SHA-256 del Markdown: 38a16a0f97505c0b405509dd3c7e3d1b50e77b89fbe3f4cb1931d3e04f602215
Tamaño del Markdown: 85,324 bytes
Conversión: representación visual completa; el Markdown original mantiene la autoridad sobre el texto fuente.

Front matter original

title	Cartridge & Cloud — Build y Versioning Guide
author	VRM Games / Blas Luis Rocha González
date	2026-07-01
lang	es-ES
document_version	1.0
status	Fuente vigente de build, versionado, empaquetado y trazabilidad
project	Cartridge & Cloud
platform	PC / Steam
engine	Unity 6.3 LTS 6000.3.18f1
render_pipeline	URP 17.3.0
application_version_reference	0.0.17
persistence_schema_reference	IntegratedGameStateSnapshot schema 2
documentary_baseline_reference	v0.6

Índice

Cartridge & Cloud — Build y Versioning Guide

- 0. Propósito
 - 0.1. Preguntas que debe responder cada build
 - 0.2. Alcance
 - 0.3. Fuera de alcance inmediato
- 1. Jerarquía de autoridad
 - 1.1. Verdad implementada frente a objetivo
 - 1.2. Fecha de la fotografía
- 2. Fotografía técnica actual
 - 2.1. Discrepancias activas
 - 2.2. Ausencia de automatización Unity
- 3. Los ejes de versión
 - 3.1. Prohibición de colapsar ejes
 - 3.2. Identificador completo de una build
- 4. SemVer adaptado al proyecto
 - 4.1. Periodo 0.x
 - 4.2. Incremento PATCH
 - 4.3. Incremento MINOR
 - 4.4. Incremento MAJOR
 - 4.5. Prerelease y metadata
- 5. Estado y ciclo de una versión
 - 5.1. Congelación
 - 5.2. Una versión, varias builds
- 6. Cambios incompatibles
 - 6.1. Clases de breaking change
 - 6.2. Procedimiento
 - 6.3. Prohibición de invalidación silenciosa
- 7. Schema de persistencia y compatibilidad
 - 7.1. Regla de incremento
 - 7.2. Matriz de compatibilidad
 - 7.3. Lectura y escritura
 - 7.4. Build candidate y saves
- 8. Versionado de contenido
 - 8.1. Objetivo
 - 8.2. Cambios peligrosos
 - 8.3. Regla
- 9. Versionado documental
 - 9.1. Baselines
 - 9.2. Cadencia
 - 9.3. Relación con una build
- 10. Repositorio, ramas y working copy
 - 10.1. Rama estable
 - 10.2. Ramas
 - 10.3. Working copy limpia
 - 10.4. SHA
 - 10.5. Tags
- 11. Tipos de build
 - 11.1. Development
 - 11.2. QA
 - 11.3. Candidato de hito
 - 11.4. Release
- 12. Build Profiles
 - 12.1. Perfil observado Windows_Development
 - 12.2. Perfiles objetivo
 - 12.3. Ownership
 - 12.4. Cambio de perfil
- 13. Escenas y orden de arranque
 - 13.1. Regla permanente

- 13.2. Estado observado
- 13.3. Objetivo Sprint 16
- 13.4. Distribución
- 13.5. Validación
- 14. Plataforma Windows x64
 - 14.1. Baseline
 - 14.2. Archivos mínimos esperados
 - 14.3. Resolución y ventana
 - 14.4. Ruta de guardado
- 15. Scripting backend y runtime
 - 15.1. Mono en desarrollo
 - 15.2. IL2CPP
 - 15.3. Prohibición
 - 15.4. Stripping
- 16. Flags de desarrollo, debugging y profiling
 - 16.1. Builds de profiling
 - 16.2. Logging
- 17. Nomenclatura de builds
 - 17.1. Formato
 - 17.2. Reglas
 - 17.3. Build ID interno
- 18. Estructura de salida
 - 18.1. Separación de staging
- 19. Metadata y manifiesto de build
 - 19.1. Campos mínimos
 - 19.2. Metadata embebida
 - 19.3. Fuente de verdad
- 20. Preflight de build
 - 20.1. Código y repositorio
 - 20.2. Unity
 - 20.3. Datos
 - 20.4. QA
 - 20.5. Resultado de preflight
- 21. Tests automatizados
 - 21.1. Antes del build
 - 21.2. Después del build
 - 21.3. Fallos
- 22. Build manual en Unity
 - 22.1. Fallo de build
 - 22.2. Ejecución externa
- 23. Smoke test externo
- 24. Golden Path de candidato
 - 24.1. Campaña de siete días
 - 24.2. Criterio
- 25. Player.log y diagnóstico
 - 25.1. Cuándo es obligatorio
 - 25.2. Revisión
 - 25.3. Preservación
 - 25.4. Crash logs
- 26. Evidencia de build
 - 26.1. Sprint ordinario
 - 26.2. Cierre de fase/candidato
 - 26.3. Evidencia inválida
- 27. Empaquetado
 - 27.1. Procedimiento
 - 27.2. Contenido adicional por canal
 - 27.3. ZIP
- 28. Checksum e integridad
 - 28.1. SHA-256
 - 28.2. Qué demuestra
 - 28.3. Firma
- 29. Build record
 - 29.1. Datos no disponibles
- 30. Changelog
 - 30.1. Categorías
 - 30.2. Reglas

- 30.3. Changelog de baseline
- 31. Release notes
 - 31.1. Internas
 - 31.2. Públicas
- 32. Known issues y defectos
 - 32.1. Severidades
 - 32.2. Snapshot de defectos
 - 32.3. Retirada
- 33. Aprobación de build
 - 33.1. Estados
 - 33.2. Autoridad
 - 33.3. PASS WITH ACCEPTED DEBT
- 34. Builds reproducibles
 - 34.1. Objetivo realista
 - 34.2. Determinismo
 - 34.3. Clean build
 - 34.4. Comparación
- 35. Automatización por línea de comandos
 - 35.1. Ejecución de tests
 - 35.2. Build
 - 35.3. Argumentos
 - 35.4. Códigos de salida
- 36. Diseño de BuildCommand
 - 36.1. Responsabilidades
 - 36.2. No responsabilidades
 - 36.3. Pseudocódigo
- 37. Integración continua futura
 - 37.1. Pipeline recomendado
 - 37.2. Seguridad
 - 37.3. Caché
 - 37.4. Gate manual
 - 37.5. Retención CI
- 38. Archivo y retención

- 38.1. Clases
- 38.2. Qué archivar
- 38.3. Ubicaciones
- 39. Rollback
 - 39.1. Condiciones
 - 39.2. Rollback de código
 - 39.3. Saves
 - 39.4. Registro
- 40. Hotfix
 - 40.1. Cuándo
 - 40.2. Flujo
 - 40.3. Versionado
 - 40.4. Scope
- 41. Steam y distribución futura
 - 41.1. Estado
 - 41.2. Requisitos previos
 - 41.3. App, depots y branches
 - 41.4. Steam BuildID
 - 41.5. SteamPipe
 - 41.6. Steam Cloud
- 42. Demo
 - 42.1. Build separada
 - 42.2. Versionado
 - 42.3. Save transfer
- 43. Licencias, créditos y seguridad
 - 43.1. Licencias
 - 43.2. Secretos
 - 43.3. Escaneo
- 44. Checklist Sprint 16
 - 44.1. Entrada
 - 44.2. Perfil
 - 44.3. QA
 - 44.4. Cierre

45. Checklist Sprint 17 y H6

45.1. Sprint 17

45.2. Candidato H6

45.3. Tras PASS

46. Alpha, Beta, RC y Release

46.1. Alpha

46.2. Beta

46.3. Release Candidate

46.4. Release

47. Definition of Done de una build

48. Patrones prohibidos

49. Criterios de aceptación de la política

50. Matriz de decisión por tipo de cambio

51. Plantilla de preflight

52. Plantilla de manifest JSON

53. Plantilla de release record

54. Historial de builds verificado

54.1. Lecciones

55. Deuda y evolución recomendada

Prioridad inmediata

Antes de H6

Antes de demo/Alpha

Antes de Release

56. Taxonomía de fallos de build

56.1. Fallo de compilación C#

56.2. Fallo de importación o asset

56.3. Fallo de Scene List

56.4. Fallo de Player distinto del Editor

56.5. Fallo de empaquetado o hash

56.6. Fallo de almacenamiento

57. Entorno y hardware del build

57.1. Registro del entorno

57.2. Hora y fecha

57.3. Máquina de build

58. Procedimiento de bump de versión

58.1. Bump ordinario de sprint

58.2. Bump por hotfix

58.3. Bump de schema

58.4. Bump documental

58.5. Bump de contenido

59. Matriz de artefactos por gate

59.1. Regla de proporcionalidad

60. Escenarios de migración y actualización

60.1. Nueva instalación

60.2. Actualización compatible

60.3. Actualización con migración

60.4. Save futuro

60.5. Cambio de Application Identifier

60.6. Demo a juego completo

61. Revisión formal de un candidato

61.1. Reunión o acto de gate

61.2. Preguntas de rechazo

61.3. Cierre

62. Runbook de recuperación

62.1. La build candidata no arranca

62.2. El hash publicado no coincide

62.3. Se incluyó un secreto

62.4. Corrupción de saves tras update

62.5. Build profile dañado

62.6. Paquete o Unity cambió accidentalmente

Anexo A. Rutas relevantes

Anexo B. Valores protegidos

Anexo C. Definiciones

Anexo D. Ruta final

Anexo E. Trazabilidad de fuentes

Cartridge & Cloud — Build y Versioning Guide

0. Propósito

Este documento define la política autoritativa para identificar, producir, validar, empaquetar, archivar y distribuir builds de **Cartridge & Cloud**. También define cómo se relacionan las versiones de aplicación, documentación, persistencia, contenido, motor, paquetes, artefactos y repositorio.

La guía debe impedir cinco errores frecuentes:

1. llamar “release” a una carpeta local no identificada;
2. cambiar la versión visible sin registrar el código que la produjo;
3. tratar un cambio de schema como si fuera un simple cambio cosmético;
4. aprobar una build porque Unity mostró `Build Succeeded` sin ejecutarla fuera del Editor;
5. publicar un artefacto que no pueda asociarse de forma inequívoca a un commit, configuración, conjunto de escenas, pruebas y known issues.

La guía no convierte automáticamente el proyecto en una producción con CI, firma digital o SteamPipe. Describe la norma actual y la evolución preparada. Cuando una capacidad todavía no existe, se marca como **objetivo**, **propuesta** o **deuda**, nunca como implementación cerrada.

0.1. Preguntas que debe responder cada build

Toda build relevante debe permitir responder:

- ¿qué código exacto la produjo?;
- ¿qué versión de Unity y paquetes se usó?;
- ¿qué versión de aplicación muestra?;
- ¿qué schema de guardado entiende?;
- ¿qué escenas contiene y en qué orden?;
- ¿qué perfil y opciones de compilación se aplicaron?;
- ¿qué pruebas se ejecutaron antes y después?;
- ¿qué recorrido externo se validó?;

- ¿qué defectos permanecen abiertos?;
- ¿dónde están el ejecutable, el log y la evidencia?;
- ¿puede reproducirse o, como mínimo, auditarse?;
- ¿puede retirarse o reemplazarse sin confundir a testers o usuarios?;

0.2. Alcance

La norma cubre:

- builds locales de desarrollo;
- builds QA;
- candidatos de Sprint 16, Sprint 17 y H6;
- demo, Alpha, Beta, Release Candidate y Release futuras;
- versionado SemVer adaptado;
- schema y migraciones;
- perfiles de Unity;
- Windows x64;
- nomenclatura y estructura de salida;
- pruebas y aceptación;
- `Player.log`, crash logs y diagnóstico;
- checksums, archivo y retención;
- tags, releases y changelog;
- rollback y hotfix;
- automatización futura;
- preparación para Steam sin asumir que Steamworks esté implementado.

0.3. Fuera de alcance inmediato

No se considera actualmente implementado:

- pipeline CI de Unity;
- firma de código para Windows;
- instalador MSI/MSIX;
- Steamworks SDK;

- SteamPipe;
- depots o branches de Steam;
- actualización diferencial;
- crash reporting remoto;
- telemetría de builds;
- símbolos públicos de depuración;
- distribución para macOS, Linux o consolas;
- un perfil Release final aprobado;
- IL2CPP como backend autorizado para distribución.

Estas capacidades pueden incorporarse mediante roadmap, spike, ADR y QA específicos.

1. Jerarquía de autoridad

La guía aplica el siguiente orden:

1. `00_Enfoque_y_Alcance.md`;
2. `01_Game_Design_Document.md`;
3. `02_Vertical_Slice_Specification.md`;
4. `03_Technical_Design_Document.md`;
5. `04_Modelo_de_Datos.md`;
6. `05_UX_Flow.md`;
7. `06_Production_Roadmap_y_Sprint_Plan.md`;
8. `07_QA_Testing_Plan.md`;
9. `08_QA_Testing_Matrix.xlsx`;
10. `09_CSharp_Coding_Standards.md`;
11. `10_Unity_Project_Setup_Guide.md`;
12. esta guía consolidada;
13. `ProjectSettings` , Build Profiles, paquetes y scripts reales;
14. ADR y registros operativos;
15. Build & Versioning Guide v0.6;

16. versiones v0.5, v0.4 y v0.3.

Cuando una política histórica es menos estricta que un gate actual, prevalece el gate actual. Por ejemplo, ADR-0010 permite omitir la revisión rutinaria de `Player.log` en un sprint ordinario sin incidencias; sin embargo, Sprint 16, Sprint 17 y H6 exigen expresamente build externa y revisión del log. Para esos gates se aplica la exigencia superior.

1.1. Verdad implementada frente a objetivo

Se usan estas etiquetas:

Etiqueta	Significado
OBSERVADO	valor leído en los archivos suministrados
VALIDADO	valor respaldado por registro de ejecución
VIGENTE	norma aprobada en este documento
OBJETIVO	configuración que debe implementarse antes de un gate
DIFERIDO	previsto para una fase posterior
REQUIERE ADR	cambio estructural que no puede aplicarse informalmente

1.2. Fecha de la fotografía

La fotografía técnica corresponde al **1 de julio de 2026**. Un SHA, versión o estado no debe reutilizarse como si describiera automáticamente una working copy posterior.

2. Fotografía técnica actual

Campo	Estado observado
Proyecto	Cartridge & Cloud
Compañía	VRM Games
Unity	6000.3.18f1 (5eb53e4c07)
URP	17.3.0

Campo	Estado observado
Plataforma objetivo	Windows x64
Versión de aplicación	0.0.17
Application Identifier	com.vrmgames.cartridgeandcloud
Input	Input System (activeInputHandler: 1)
Player log	activado
Compilación determinista C#	activada
Backend de desarrollo	Mono por baseline/ausencia de override Standalone
Compresión Development	LZ4
Build Profile	Windows_Development
Escenas del perfil	Bootstrap, MainMenu, Store, TestLab
Escena objetivo del slice	StoreInitial
Resolución por defecto observada	1024 × 768
Ventana redimensionable observada	desactivada
Schema de sesión mínimo	1
Schema integrado	2
Tests documentados	1215 EditMode + 70 PlayMode = 1285 PASS
Estado	Sprints 0-15 cerrados; Sprint 16 activo; Sprint 17 pendiente

2.1. Discrepancias activas

La configuración observada todavía no representa el objetivo final de Sprint 16:

- el Build Profile incluye `Store`, no `StoreInitial`;
- la lista global también incluye `Store`;
- `TestLab` está incluido en el perfil de desarrollo;
- no se encontró un script de build del Player en `Assets` o `Tools`;
- la build se produce actualmente mediante flujo manual de Unity;
- no existe pipeline CI aportado;
- no existe perfil QA/H6/Release separado;

- no existe metadata de build embebida confirmada;
- `buildNumber.Standalone` permanece en `0`;
- la resolución `1024 × 768` no debe interpretarse como objetivo comercial aprobado;
- no hay evidencia de firma, SteamPipe o distribución pública.

Ninguna discrepancia debe “corregirse” fuera de su gate. La migración `Store → StoreInitial` se realiza después de conectar y validar la nueva escena, no antes.

2.2. Ausencia de automatización Unity

La inspección del paquete suministrado no encontró una clase Editor dedicada a producir el Player ni un script de línea de comandos para Unity. `Tools/Blender/.../build_all.py` pertenece al kit de Blender y no es un pipeline del ejecutable. Por tanto, los ejemplos de automatización incluidos en esta guía son **diseño de referencia**, no descripción de una herramienta ya disponible.

3. Los ejes de versión

Cartridge & Cloud no tiene una única “versión”. Tiene varios ejes que deben registrarse por separado.

Eje	Ejemplo	Autoridad	Qué identifica
Aplicación	<code>0.0.17</code>	<code>PlayerSettings.bundleVersion</code>	evolución jugable/técnica
Build	<code>Build003</code>	registro de artefacto	iteración concreta de una misma versión
Git	SHA de 40 caracteres	repositorio	código y assets exactos
Schema	<code>2</code>	snapshot/codec	forma persistente compatible
Contenido	<code>content manifest</code> futuro	catálogo	conjunto de definiciones/assets
Documento	<code>1.0 / baseline v0.6</code>	front matter/baseline	norma o fotografía documental
Unity	<code>6000.3.18f1</code>	<code>ProjectVersion.txt</code>	toolchain
Paquetes	URP 17.3.0, etc.	manifest/lock	dependencias resueltas
Steam	BuildID futuro	Steamworks	subida aprobada por Valve

3.1. Prohibición de colapsar ejes

No se debe afirmar:

- “schema 0.0.17”;
- “build v0.6” sin aclarar si v0.6 es documentación;
- “release d54316” como sustituto de una versión de aplicación;
- “Steam build 0.3.0” sin registrar el BuildID real;
- “última build” sin Build ID, fecha, SHA y ubicación.

3.2. Identificador completo de una build

Una build queda identificada por la tupla:

```
ApplicationVersion  
+ BuildIteration  
+ CommitSHA  
+ UnityVersion  
+ BuildProfile  
+ Platform/Architecture  
+ SchemaVersion  
+ BuildDateUTC
```

Para un candidato de hito se añaden:

```
ArtifactSHA256  
+ TestReport  
+ PlayerLogReview  
+ KnownIssuesSnapshot  
+ GateDecision
```

4. SemVer adaptado al proyecto

Se usa `MAJOR.MINOR.PATCH`, sin prefijo `v` dentro de `PlayerSettings`. El prefijo `v` se reserva para tags o nombres humanos cuando resulte útil.

4.1. Periodo 0.x

Mientras `MAJOR = 0`, el proyecto sigue en desarrollo previo al contrato comercial estable. Esto no autoriza cambios arbitrarios: cada cambio incompatible de datos, experiencia o distribución debe documentarse.

La secuencia inicial observada usa el patch para representar el cierre de sprints:

Sprint	Versión	Capacidad cerrada
0	0.0.1	fundación y primera build
1	0.0.2	Bootstrap y flujo de escenas
2	0.0.3	sesión, slots y save skeleton
3	0.0.4	movimiento, cámara e input
4	0.0.5	grid y placement
5	0.0.6	shell de tienda y acceso
6	0.0.7	productos e inventario
7	0.0.8	pedidos y recepción
8	0.0.9	displays y reposición
9	0.0.10	perfiles y spawning de clientes
10	0.0.11	shopping y reservas
11	0.0.12	cola y checkout
12	0.0.13	ciclo diario y cierre
13	0.0.14	economía y resultados
14	0.0.15	save/load integrado
15	0.0.16	integración UI/UX
16	0.0.17	presentación representativa en curso
17	por congelar	estabilización y H6

La versión de Sprint 17 no se inventa en esta guía. Debe congelarse al abrir el sprint y comprobar que no colisiona con hotfixes o builds de Sprint 16.

4.2. Incremento PATCH

Se incrementa PATCH para:

- cierre ordinario de sprint dentro de la línea `0.0.x`;
- hotfix que no añade un bloque de producto;
- corrección de build distribuida manteniendo compatibilidad;
- revisión interna que cambia código, assets o datos ejecutables.

Un cambio puramente documental no obliga a modificar `bundleVersion`, aunque sí modifica la versión o hash documental.

4.3. Incremento MINOR

Durante `0.x`, MINOR se reserva para una transición de madurez o contrato de producto, por ejemplo:

- vertical slice aprobado;
- MVP de tienda aprobado;
- demo pública;
- Alpha funcional;
- Beta feature complete.

La decisión debe registrarse en el gate. No se aplica solo porque el número de patch resulte alto.

4.4. Incremento MAJOR

`1.0.0` corresponde al primer release comercial estable aprobado. Un `2.0.0` futuro indicaría una ruptura de contrato pública de gran escala, no una migración interna cualquiera.

4.5. Prerelease y metadata

Para nombres de artefacto o canales puede usarse:

```
0.3.0-h6.1  
0.4.0-alpha.2  
0.4.0-beta.5  
1.0.0-rc.1
```

Unity `bundleVersion` debe mantenerse simple y verificarse en el Player. La metadata adicional puede vivir en un manifiesto de build y en el nombre del artefacto.

5. Estado y ciclo de una versión

Estado	Significado
PROPOSED	versión candidata no congelada
FROZEN	número reservado para un sprint/gate
BUILT	existe al menos una build identificada
VALIDATING	artefacto bajo QA
ACCEPTED	gate aprobado
SUPERSEDED	reemplazada por otra build/versión
WITHDRAWN	retirada por defecto o error de publicación
RELEASED	distribuida al canal previsto

5.1. Congelación

Antes de producir una build de cierre:

1. confirmar objetivo y gate;
2. resolver la versión propuesta;
3. actualizar `PlayerSettings.bundleVersion`;
4. registrar schema compatible;
5. comprobar que el changelog contiene solo cambios verificables;
6. impedir cambios de alcance no relacionados;
7. registrar el SHA que se pretende validar.

5.2. Una versión, varias builds

Una versión puede tener varias iteraciones:


```
0.0.17 / Build001 – preintegración representativa  
0.0.17 / Build002 – primera conexión de StoreInitial  
0.0.17 / Build003 – corrección de clic sobre UI  
0.0.17 / Build004 – candidato de cierre de Sprint 16
```

No se incrementa la versión por cada intento fallido. Sí se incrementa el contador de build cuando se conserva o comunica el artefacto.

6. Cambios incompatibles

Un cambio es incompatible si puede impedir que una build anterior o posterior interprete estado, contenido, comandos o distribución de forma segura.

6.1. Clases de breaking change

- cambio de schema persistente;
- cambio semántico de un campo existente;
- eliminación o reutilización de un ID estable;
- modificación del ownership de inventario o reservas;
- cambio de unidad monetaria;
- cambio del orden o identidad de escenas requeridas;
- cambio de Application Identifier o ruta de guardado;
- cambio de backend o stripping que elimina código requerido;
- cambio de catálogo que deja referencias persistentes huérfanas;
- cambio de contrato de input o cierre que hace una partida no operable;
- cambio de API pública de herramientas de build consumidas por CI.

6.2. Procedimiento

Todo breaking change requiere:

1. descripción y motivación;

2. impacto en saves, tests, builds y rollback;
3. nueva versión de schema cuando corresponda;
4. migrador o declaración explícita de incompatibilidad;
5. fixtures de la versión anterior;
6. prueba de upgrade;
7. prueba de rechazo de versiones futuras;
8. actualización de known issues y release notes;
9. ADR si afecta arquitectura o distribución;
10. plan de recuperación.

6.3. Prohibición de invalidación silenciosa

No se puede borrar, ignorar o reemplazar un save incompatible sin informar. En builds internas se puede declarar una ruptura controlada, pero debe aparecer en el registro antes de entregar el artefacto.

7. Schema de persistencia y compatibilidad

La baseline distingue:

- `GameSessionSnapshot` schema 1;
- `IntegratedGameStateSnapshot` schema 2.

El schema integrado es la ruta autoritativa objetivo del slice. La ausencia actual de placement dinámico en schema 2 se trata como deuda conocida, no como compatibilidad completa.

7.1. Regla de incremento

Se incrementa schema cuando cambia la estructura o semántica persistente. No se incrementa por:

- refactor interno sin cambio de serialización;
- corrección de UI;
- cambio visual no persistente;
- aumento de cobertura de tests;

- cambio documental.

7.2. Matriz de compatibilidad

Build	Schema mínimo legible	Schema escrito	Observación
pre-S14	variable/esqueleto	anterior	no usar como contrato actual
0.0.15+	según codec validado	2	snapshot integrado
futuro schema 3	1/2 si hay migrador	3	requiere fixtures y gate

La tabla debe actualizarse con evidencia real antes de una distribución externa.

7.3. Lectura y escritura

- una build no debe escribir un schema que no pueda validar;
- una build que detecte schema futuro debe rechazarlo sin mutación;
- la lectura debe completarse en memoria antes de aplicar estado;
- la escritura usa temporal, validación, promoción y backup;
- el primario corrupto no debe destruir un backup válido;
- un downgrade no se presupone compatible;
- probar “carga sin excepción” no basta: debe verificarse equivalencia observable.

7.4. Build candidate y saves

Antes de aprobar un candidato de hito:

- crear save nuevo;
- guardar y cargar;
- cerrar proceso y continuar;
- probar backup;
- probar corrupción controlada;
- probar schema futuro no soportado;
- comprobar tres slots;
- verificar inventario, reservas, checkout, día y economía;

- registrar versión y schema en evidencia.

8. Versionado de contenido

Los assets y catálogos pueden cambiar sin modificar la forma del snapshot, pero aun así romper una partida. Por ello se prepara un `content manifest version` independiente.

8.1. Objetivo

El manifiesto futuro debe identificar:

- productos;
- mobiliario;
- proveedores;
- perfiles de clientes;
- audio/VFX requeridos;
- escenas y catálogos;
- definiciones de balance;
- hashes o versión del conjunto.

8.2. Cambios peligrosos

- borrar un ID persistido;
- reasignar un ID a otro concepto;
- cambiar una huella de mueble persistido;
- retirar un producto con stock existente;
- renombrar una escena sin migrar referencias;
- modificar una definición de precio que deba permanecer congelada en una transacción.

8.3. Regla

Los IDs se deprecian antes de eliminarse. Un catálogo nuevo debe poder:

- resolver el ID antiguo;
 - migrarlo;
 - sustituirlo por fallback explícito;
 - o rechazar la carga con diagnóstico claro.
-

9. Versionado documental

La versión documental no se sincroniza mecánicamente con la aplicación.

9.1. Baselines

Una baseline documental representa una fotografía aprobada. No debe reescribirse para ocultar historia. Las correcciones posteriores generan:

- nueva versión;
- changelog;
- hash;
- o registro operativo complementario.

9.2. Cadencia

- registros operativos: por sprint o ejecución;
- baseline oficial: por cierre de fase;
- baseline intermedia: solo ante cambio material de arquitectura, plataforma, tooling o dirección;
- esta guía: actualizar cuando cambia la política de build/versionado.

9.3. Relación con una build

Un candidato de hito debe enlazar la baseline documental que describe su comportamiento. Una baseline no prueba que el ejecutable corresponda a ella; la asociación exige SHA, registro de build y evidencia.

10. Repositorio, ramas y working copy

10.1. Rama estable

`main` es la referencia integrada. Una build candidata debe producirse desde:

- commit publicado y limpio;
- o commit candidato claramente identificado antes de la validación final.

No se aprueba un hito desde cambios locales no registrados.

10.2. Ramas

Convenciones permitidas:

```
feature/<tema>
fix/<tema>
docs/<tema>
release/<version>
hotfix/<version>--<tema>
```

En desarrollo individual pueden ser breves. No deben crear una falsa sensación de revisión: antes de integrar siguen siendo obligatorios compilación, tests y aceptación.

10.3. Working copy limpia

Preflight mínimo:

```
git status
```

Debe distinguirse:

- cambios intencionales incluidos;
- archivos generados ignorados;
- evidencia aún no publicada;

- cambios accidentales en escenas, settings o `.meta`;
- subidas incompletas.

10.4. SHA

Registrar SHA completa de 40 caracteres. Una abreviatura puede mostrarse en UI, pero el build record conserva la completa.

10.5. Tags

Los tags representan hitos, no cada sprint. Formato recomendado:

```
v0.0.1-project-foundation  
v0.3.0-vertical-slice  
v0.4.0-alpha.1  
v1.0.0
```

Un tag:

- apunta al commit validado;
- no se mueve después de publicar;
- se crea después del PASS del gate;
- no se crea para un artefacto fallido;
- incluye release record cuando se distribuye.

11. Tipos de build

Tipo	Propósito	Audiencia	Artefacto retenido
Editor Test	desarrollo inmediato	desarrollador	no
Development	smoke y diagnóstico	interno	opcional
QA	regresión y campañas	QA interno/testers	sí durante campaña
H6 Candidate	aprobación del vertical slice	gate interno	sí

Tipo	Propósito	Audiencia	Artefacto retenido
Private Tester	prueba controlada	testers seleccionados	sí
Demo	experiencia pública limitada	público	sí
Alpha	feature coverage	interno/externo limitado	sí
Beta	estabilización y contenido	testers	sí
RC	candidato a lanzamiento	release team	sí
Release	distribución pública	usuarios	sí y permanente
Hotfix	corrección urgente	canal afectado	sí

11.1. Development

- puede incluir `TestLab`;
- usa Mono y LZ4 mientras siga aprobada la baseline;
- permite Development Build;
- no requiere ZIP para cada iteración;
- debe ejecutarse externamente cuando cambia comportamiento Player;
- no se comunica como demo.

11.2. QA

- perfil separado objetivo;
- escenas de producción, sin herramientas innecesarias;
- logging suficiente;
- build identificada;
- resultados y defectos asociados;
- puede conservar símbolos privados según la investigación.

11.3. Candidato de hito

- checkout limpio;
- versión congelada;
- pruebas completas;

- Golden Path;
- `Player.log` revisado;
- checksum;
- paquete de evidencia;
- known issues congelados;
- decisión formal.

11.4. Release

No es simplemente un Development Build con el flag desmarcado. Requiere decisiones sobre:

- backend;
- stripping;
- símbolos;
- logging;
- crash reporting;
- firma;
- Steam;
- licencias;
- privacidad;
- soporte y rollback.

12. Build Profiles

12.1. Perfil observado `Windows_Development`

Campo	Valor
Asset	<code>Assets/_Project/Settings/BuildProfiles/Windows_Development.asset</code>
Target	Windows Standalone
Arquitectura	Intel 64-bit

Campo	Valor
Development	activado
Profiler autoconnect	desactivado
Deep Profiling	desactivado
Script Debugging	desactivado
Wait for Managed Debugger	desactivado
Copy PDB	desactivado
Create Solution	desactivado
Compression	LZ4
Override scene list	activado

12.2. Perfiles objetivo

```
Windows_Development
Windows_QA
Windows_H6_Candidate
Windows_Demo
Windows_Release
```

No es obligatorio crear todos ahora. Cada perfil se crea al abrir el gate que lo necesita.

12.3. Ownership

Los Build Profiles:

- se versionan;
- se revisan como configuración crítica;
- no se editan durante un build sin registrar el cambio;
- no deben depender de una lista global diferente sin explicación;
- deben tener un test o checklist de escenas;
- no contienen secretos.

12.4. Cambio de perfil

Cambiar backend, escenas, stripping, arquitectura, compression, development flags o player settings de distribución exige:

1. diff del asset;
2. motivo;
3. build antes/después;
4. regresión;
5. actualización de esta guía o ADR si es estructural.

13. Escenas y orden de arranque

13.1. Regla permanente

`Bootstrap` es la primera escena de toda build jugable.

13.2. Estado observado

```
0 - Bootstrap
1 - MainMenu
2 - Store
3 - TestLab
```

El orden aparece tanto en la lista global como en `Windows_Development`.

13.3. Objetivo Sprint 16

Después de conectar `StoreInitial` y validar runtime:

```
0 - Bootstrap
1 - MainMenu
```

```
2 - StoreInitial
3 - TestLab      # solo Development
```

La escena `Store` se conserva temporalmente como fallback o referencia hasta cerrar la migración, pero no debe permanecer como destino ambiguo del flujo aprobado.

13.4. Distribución

Para QA de hito, demo, Alpha, Beta, RC y Release:

```
0 - Bootstrap
1 - MainMenu
2 - StoreInitial
```

`TestLab` se excluye salvo que exista una razón interna explícita. No debe ser accesible por el usuario final.

13.5. Validación

- cada ruta existe;
- cada GUID es correcto;
- no hay escenas deshabilitadas inesperadas;
- MainMenu carga el destino previsto;
- Continue restaura la escena correcta;
- volver al menú no duplica `ApplicationRoot`;
- Quit finaliza el proceso;
- no se inicia directamente desde una escena de contenido en la build.

14. Plataforma Windows x64

14.1. Baseline

- target: Standalone Windows;

- arquitectura: x86_64;
- ejecutable: `CartridgeAndCloud.exe`;
- Application Identifier: `com.vrmgames.cartridgeandcloud`;
- ruta de datos Unity asociada a compañía/producto;
- Player log activado.

14.2. Archivos mínimos esperados

```
CartridgeAndCloud.exe
CartridgeAndCloud_Data/
UnityPlayer.dll
UnityCrashHandler64.exe
```

La lista exacta puede variar con backend, Unity y plugins. La validación no debe depender de una lista histórica rígida si el toolchain cambia de forma aprobada.

14.3. Resolución y ventana

El proyecto observa `1024 × 768` y ventana no redimensionable. Antes de H6 deben verificarse al menos `1280 × 720`, `1920 × 1080` y `2560 × 1440` conforme a QA. Cambiar los valores por defecto requiere validar:

- HUD;
- modales;
- escala UI;
- cámara;
- placement;
- localización ES/EN;
- modo pantalla completa/ventana;
- persistencia de opciones si existe.

14.4. Ruta de guardado

La ruta depende de `companyName` y `productName`. Cambiar cualquiera puede aparentar pérdida de saves. Requiere ADR, migración y prueba en una instalación existente.

15. Scripting backend y runtime

15.1. Mono en desarrollo

Mono está aprobado para iteración de desarrollo. Ventajas dentro de la baseline:

- ciclo de build rápido;
- diagnóstico sencillo;
- coherencia con registros históricos;
- menor coste durante integración.

15.2. IL2CPP

IL2CPP está diferido. Antes de adoptarlo:

- crear ADR;
- medir tiempo de build;
- validar stripping;
- probar serialización y reflection;
- probar plugins;
- ejecutar suite completa;
- comparar rendimiento;
- revisar stack traces y símbolos;
- validar Steam Overlay cuando exista;
- conservar una ruta de rollback.

15.3. Prohibición

No cambiar a IL2CPP “porque es Release” sin realizar el gate. Un backend distinto puede descubrir fallos que no aparecen en Mono.

15.4. Stripping

El stripping actual no constituye una política Release cerrada. Cualquier aumento requiere pruebas sobre:

- serialización;
- tipos creados dinámicamente;
- JSON;
- reflection;
- ScriptableObjects;
- localización;
- addressables si se incorporan;
- código de plugins.

16. Flags de desarrollo, debugging y profiling

Opción	Development	QA	Candidate	Release
Development Build	sí	según campaña	no por defecto	no
Script Debugging	no por defecto	solo investigación	no	no
Autoconnect Profiler	no	solo sesión dedicada	no	no
Deep Profiling	no	solo build especial	no	no
Player Log	sí	sí	sí	política por decidir
PDB/símbolos	no baseline	privados si se aprueba	privados	archivo seguro

16.1. Builds de profiling

Una build de profiling se etiqueta explícitamente. No puede usarse para medir rendimiento final si Deep Profiling altera de forma material el resultado.

16.2. Logging

- no registrar secretos o datos personales;

- evitar spam por frame;
- incluir contexto e ID estable;
- diferenciar info, warning y error;
- no ocultar error con `try/catch` vacío;
- una build candidata no puede presentar errores recurrentes ignorados.

17. Nomenclatura de builds

17.1. Formato

```
CAC_v<appVersion>_<milestone>_<type>_Windows_x64_<YYYY-MM-DD>_Build<NNN>
```

Ejemplos:

```
CAC_v0.0.17_Sprint16_Dev_Windows_x64_2026-07-01_Build004  
CAC_v0.3.0_H6_Candidate_Windows_x64_2026-07-XX_Build001  
CAC_v0.4.0_Alpha_Private_Windows_x64_2026-XX-XX_Build002  
CAC_v1.0.0_RC_Windows_x64_2027-XX-XX_Build003
```

17.2. Reglas

- ASCII, sin espacios;
- fecha de producción, no de publicación tardía;
- contador con tres dígitos;
- no usar “Final” salvo release aprobado; preferir `Candidate` o `BuildNNN`;
- no sobrescribir una carpeta ya validada;
- ZIP conserva el mismo basename;
- checksum usa `<basename>.sha256`;
- build record usa `<basename>_Build_Record.md` o ID equivalente.

17.3. Build ID interno

Formato recomendado:

```
S16-WIN-DEV-004  
S17-WIN-QA-001  
H6-WIN-CAND-001  
RC1-WIN-REL-001
```

El Build ID es corto; el nombre del artefacto es descriptivo. Ambos aparecen en el manifiesto.

18. Estructura de salida

```
Builds/  
└─ Windows/  
    │   └─ Development/  
    │       └─ <BuildName>/  
    │   └─ QA/  
    │       └─ <BuildName>/  
    │   └─ Candidates/  
    │       └─ <BuildName>/  
    │   └─ Demo/  
    │   └─ Release/  
    └─ Archive/
```

Builds/ permanece fuera de Git. Los registros, checksums, scripts y manifiestos sí se versionan en documentación o herramientas.

18.1. Separación de staging

Para candidatos:

```
Artifacts/  
└─ player/  
└─ logs/  
└─ tests/  
└─ evidence/
```

```
├─ manifests/  
└─ checksums/
```

No empaquetar:

- saves personales;
- `Player.log` de otra build;
- capturas no relacionadas;
- credenciales;
- archivos de Unity Editor;
- `Library`;
- source code salvo paquete acordado;
- builds anteriores dentro del ZIP.

19. Metadata y manifiesto de build

Cada build retenida debe tener un manifiesto legible por humanos y, cuando se automatice, por máquinas.

19.1. Campos mínimos

```
build_id: S16-WIN-DEV-004  
application_version: 0.0.17  
build_iteration: 4  
commit_sha: <40-char-sha>  
build_date_utc: 2026-07-01T00:00:00Z  
unity_version: 6000.3.18f1  
build_profile: Windows_Development  
platform: Windows  
architecture: x86_64  
scripting_backend: Mono  
compression: LZ4  
development_build: true  
schema_version: 2  
content_version: unversioned  
scenes:  
  - Bootstrap  
  - MainMenu  
  - StoreInitial
```

```
test_summary:  
  editmode: 1215/1215  
  playmode: 70/70  
gate: Sprint16  
status: candidate
```

19.2. Metadata embebida

Objetivo futuro: exponer en una pantalla diagnóstica o log:

- versión;
- Build ID;
- SHA abreviada;
- schema;
- canal;
- fecha UTC.

No exponer rutas locales, tokens o datos sensibles.

19.3. Fuente de verdad

La metadata no debe escribirse manualmente en múltiples lugares. La automatización futura debe derivar los campos desde:

- `PlayerSettings.bundleVersion`;
- Git;
- build config;
- constantes de schema;
- argumentos del pipeline.

20. Preflight de build

20.1. Código y repositorio

- working copy revisada;

- commit/branch correctos;
- compilación sin errores;
- warnings nuevos clasificados;
- conflictos resueltos;
- versión congelada;
- changelog actualizado;
- no hay cambios accidentales en settings o `.meta`.

20.2. Unity

- editor `6000.3.18f1`;
- paquetes resueltos;
- consola limpia;
- plataforma Windows activa;
- perfil correcto;
- escenas correctas;
- `Bootstrap` índice 0;
- `StoreInitial` en el perfil cuando corresponda;
- `TestLab` excluido de distribución;
- Application Identifier correcto;
- Player log activo para gates internos;
- espacio en disco suficiente.

20.3. Datos

- catálogos validan;
- IDs duplicados: 0;
- referencias faltantes: 0;
- schema definido;
- migradores presentes si cambió schema;
- fixtures de saves disponibles;
- datos ES/EN completos para el alcance;
- no hay marcas o assets no autorizados.

20.4. QA

- tests dirigidos pasan;
- suite completa según gate;
- defectos S0/S1: 0 para candidato;
- S2 clasificados;
- matriz actualizada;
- hardware y entorno definidos;
- recorrido manual preparado;
- ruta de `Player.log` conocida;
- evidencia y naming preparados.

20.5. Resultado de preflight

Estados:

```
PASS  
PASS WITH OBSERVATION  
BLOCKED  
FAIL
```

`PASS WITH OBSERVATION` no permite ignorar una condición obligatoria. Solo documenta una observación no bloqueante.

21. Tests automatizados

21.1. Antes del build

Para Sprint 16/17/H6:

- EditMode completo;
- PlayMode completo;
- tests de escena/autoría aplicables;

- tests de catálogo;
- tests de persistencia;
- targeted tests de la corrección actual.

La baseline documentada es `1215 + 70 = 1285 PASS`, pero el número futuro puede crecer. El gate se refiere a todos los tests descubiertos esperados, no a conservar exactamente 1285.

21.2. Después del build

No se repite necesariamente toda la suite de Editor después de cada build ordinaria. En un cierre de fase se recomienda:

- confirmar que el build no modificó assets/versionados;
- ejecutar smoke externo;
- volver a ejecutar regresión si se aplicó un hotfix durante el build;
- registrar cualquier diferencia Editor/Player.

21.3. Fallos

- no reconstruir repetidamente hasta obtener verde sin investigar;
- guardar XML/log cuando exista;
- crear defecto si el fallo no se resuelve inmediatamente;
- distinguir test flaky de producto defectuoso;
- no excluir un test para aprobar el gate sin decisión formal.

22. Build manual en Unity

Procedimiento vigente mientras no exista automatización:

1. abrir el proyecto con la versión exacta;
2. esperar importación y compilación completas;
3. revisar consola;
4. ejecutar tests requeridos;

5. abrir Build Profiles;
6. seleccionar `Windows_Development` o el perfil del gate;
7. verificar escenas y flags;
8. comprobar `PlayerSettings.bundleVersion`;
9. seleccionar carpeta nueva con naming aprobado;
10. ejecutar Build, no sobrescribir un candidato previo;
11. guardar el log del Editor si falla;
12. confirmar archivos de salida;
13. ejecutar el `.exe` fuera del Editor;
14. realizar smoke o Golden Path;
15. cerrar normalmente;
16. revisar `Player.log` según gate;
17. registrar resultado;
18. empaquetar y calcular hash cuando proceda.

22.1. Fallo de build

Registrar:

- hora;
- versión;
- perfil;
- último SHA;
- mensaje principal;
- stack trace;
- ruta del Editor log;
- cambios desde el último PASS;
- corrección;
- número del nuevo intento.

Un intento fallido no se borra del conocimiento del sprint si reveló una dependencia estructural.

22.2. Ejecución externa

La prueba debe iniciarse desde el explorador o terminal, no mediante “Build And Run” exclusivamente. Esto reduce dependencia del Editor y valida el artefacto entregable.

23. Smoke test externo

Smoke mínimo de una build de desarrollo:

1. proceso arranca;
2. no aparece crash dialog;
3. Bootstrap carga MainMenu;
4. MainMenu responde;
5. se puede entrar en la tienda objetivo;
6. jugador y cámara responden;
7. UI no bloquea el flujo;
8. volver al menú funciona;
9. Quit cierra el proceso;
10. no quedan procesos huérfanos.

Si el sprint afecta sistemas concretos, se añade un smoke dirigido:

- placement;
 - pedidos;
 - clientes;
 - checkout;
 - cierre de día;
 - save/load;
 - StoreInitial;
 - localización;
 - rendimiento.
-

24. Golden Path de candidato

El candidato de Sprint 16/17/H6 debe validar:

```
EXE limpio
→ Bootstrap
→ MainMenu
→ crear/continuar slot
→ StoreInitial
→ movimiento/cámara
→ Operations sin propagación al mundo
→ pedido
→ recepción
→ almacenamiento
→ display/reposición
→ precio
→ abrir tienda
→ cliente/reserva
→ cola/checkout
→ cierre
→ resultados
→ guardado
→ salir
→ continuar
→ equivalencia de estado
```

24.1. Campaña de siete días

Para H6:

- repetir operación durante siete días lógicos;
- alternar pedidos y ventas;
- comprobar liquidez;
- guardar/cargar entre días;
- revisar crecimiento de logs y memoria;
- verificar ausencia de duplicaciones;
- reconciliar ledger y resultados;
- conservar al menos el save inicial y final como evidencia.

24.2. Criterio

Un Golden Path no “pasa” por llegar a resultados si hubo:

- errores recurrentes en log;
- stock negativo;
- venta duplicada;
- pérdida de save;
- ruta bloqueada;
- UI con input fantasma;
- recuperación manual no prevista;
- reinicio necesario para continuar.

25. `Player.log` y diagnóstico

Ruta típica validada en Windows:

```
%USERPROFILE%\AppData\LocalLow\VRM Games\Cartridge & Cloud\Player.log
```

25.1. Cuándo es obligatorio

Siempre para:

- Sprint 16 post-integración;
- Sprint 17;
- H6;
- demo;
- Alpha/Beta/RC/Release;
- crash;
- fallo de arranque;
- excepción no controlada;

- carga de escena fallida;
- discrepancia Editor/Player;
- cambio de plugins, backend o persistencia de bajo nivel.

25.2. Revisión

Buscar y clasificar:

- exceptions;
- assertions;
- missing references;
- scene load failures;
- serialization errors;
- shader fallbacks;
- audio errors;
- warnings repetidos;
- stack traces;
- shutdown anómalo;
- versión/build metadata.

25.3. Preservación

Copiar el log inmediatamente después del recorrido y renombrar:

```
<BuildID>_Player.log
```

No sobrescribirlo ejecutando otra build antes de archivarlo.

25.4. Crash logs

Si existe crash:

- conservar `Player.log`;
- conservar crash dump/report;

- registrar hardware;
- no aprobar build;
- intentar reproducir con misma build;
- vincular defecto S0/S1 según impacto.

26. Evidencia de build

26.1. Sprint ordinario

Mínimo:

- versión;
- SHA;
- plataforma;
- perfil/tipo;
- resultado;
- ejecución externa;
- alcance del smoke;
- defectos;
- observaciones.

Según ADR-0010, no requiere por defecto ZIP, hash, tag o release.

26.2. Cierre de fase/candidato

Obligatorio:

- carpeta de Player;
- ZIP u otro paquete aprobado;
- SHA-256;
- build record;
- test reports;

- `Player.log`;
- Golden Path;
- known issues;
- hardware y configuración;
- versión y SHA;
- manifest;
- capturas/vídeo;
- decisión de gate;
- tag/release cuando se publique.

26.3. Evidencia inválida

No basta:

- una captura de “Build Succeeded”;
- un mensaje de chat;
- “funciona en mi PC”;
- un ZIP sin SHA o versión;
- un log sin Build ID;
- una ruta local que ya no existe;
- un tag creado antes de la aceptación.

27. Empaquetado

27.1. Procedimiento

1. cerrar el Player;
2. comprobar que la carpeta no contiene saves/logs personales;
3. verificar ejecutable y `_Data`;
4. copiar manifest y README si el canal lo exige;
5. comprimir desde la carpeta padre;

6. no incluir otra capa de carpeta accidental;
7. probar extracción en ruta distinta;
8. ejecutar desde la extracción;
9. calcular SHA-256;
10. registrar tamaño en bytes y formato humano.

27.2. Contenido adicional por canal

Canal	Contenido adicional
QA	instrucciones, known issues, formulario de feedback
Demo	EULA/avisos, créditos, controles, soporte
RC	release notes, third-party notices, checksums
Release	paquete aprobado por tienda, legal y soporte

27.3. ZIP

ZIP es apropiado para artefactos internos y testers. No presupone el formato final de Steam, que usa depots/manifests propios.

28. Checksum e integridad

28.1. SHA-256

Para artefactos retenidos:

```
<sha256> <filename.zip>
```

El hash se calcula después del empaquetado definitivo. Cualquier modificación exige nuevo hash y, normalmente, nueva Build iteration.

28.2. Qué demuestra

SHA-256 demuestra identidad del archivo, no:

- calidad;
- ausencia de malware;
- autoría legal;
- compatibilidad;
- aprobación de gate.

28.3. Firma

La firma de código está diferida. Antes de release debe decidirse:

- certificado;
- custodia;
- timestamp;
- proceso automatizado;
- renovación;
- qué binarios se firman;
- comportamiento de rollback.

Nunca almacenar la clave privada en el repositorio.

29. Build record

Plantilla mínima:

```
# <Build ID> – Build Record

## Identificación
- Versión:
- Iteración:
- SHA:
- Fecha UTC:
```

```
- Unity:
- Perfil:
- Plataforma/arquitectura:
- Backend:
- Schema:

## Escenas

## Preflight

## Pruebas automatizadas

## Build
- Resultado:
- Duración:
- Tamaño carpeta:
- Tamaño paquete:
- SHA-256:

## Ejecución externa

## Player.log

## Golden Path

## Defectos / known issues

## Artefactos

## Decisión
PASS / PASS WITH ACCEPTED DEBT / FAIL
```

29.1. Datos no disponibles

Si un dato no se registró, escribir **No registrado**, no inventar. Los registros Sprints 9-14 aplican correctamente esta regla para ruta, tamaño, hash o log exportado.

30. Changelog

30.1. Categorías

```
Added
Changed
Fixed
Removed
Deprecated
Security
Known Issues
Save Compatibility
```

30.2. Reglas

- describir resultado observable;
- no incluir trabajo no integrado;
- separar correcciones de cambios de balance;
- señalar incompatibilidad de saves;
- no prometer roadmap;
- incluir defectos conocidos relevantes;
- enlazar IDs internos cuando proceda;
- no copiar mensajes de commit sin editar.

30.3. Changelog de baseline

El changelog documental registra cambios de autoridad y estado, por ejemplo:

- v0.3 → v0.4: Sprint 0 validado;
- v0.4 → v0.5: Sprints 1–5 y `0.0.6`;
- v0.5 → v0.6: Sprints 6–15, Sprint 16 y `0.0.17`.

No debe confundirse con release notes para jugadores.

31. Release notes

31.1. Internas

Incluyen:

- objetivo del build;
- alcance;
- cambios;
- áreas de riesgo;
- instrucciones de prueba;
- known issues;
- compatibilidad de saves;
- cómo enviar feedback;
- Build ID y hash.

31.2. Públicas

Deben:

- usar lenguaje comprensible;
 - evitar detalles internos innecesarios;
 - no exponer vulnerabilidades explotables antes de corregirlas;
 - respetar marketing y legal;
 - indicar limitaciones de demo;
 - no anunciar funciones no comprometidas.
-

32. Known issues y defectos

32.1. Severidades

Severidad	Consecuencia para build
S0	build rechazada; corrupción/pérdida/bloqueo total
S1	build rechazada; Golden Path o capacidad principal imposible
S2	corregir o aceptar formalmente según gate
S3	puede diferirse con documentación
S4	polish menor

32.2. Snapshot de defectos

Cada candidato congela:

- defectos abiertos;
- severidad;
- workaround;
- impacto;
- owner;
- decisión;
- fecha.

Una lista viva posterior no sustituye al snapshot asociado al artefacto.

32.3. Retirada

Una build distribuida se retira si:

- corrompe saves;
- contiene S0/S1 no conocido;
- no corresponde al SHA declarado;
- incluye asset o licencia no autorizada;

- expone secreto;
- carga escena incorrecta;
- no puede actualizarse de forma segura.

33. Aprobación de build

33.1. Estados

```
DRAFT
BUILT
SMOKE PASS
QA IN PROGRESS
CANDIDATE
PASS
PASS WITH ACCEPTED DEBT
FAIL
WITHDRAWN
SUPERSEDED
```

33.2. Autoridad

En producción individual, la misma persona puede construir y aprobar, pero debe separar mentalmente los roles:

- builder: produce artefacto;
- QA: ejecuta evidencia;
- product owner: decide gate;
- release owner: publica.

La separación documental reduce auto-confirmación y pérdidas de contexto.

33.3. PASS WITH ACCEPTED DEBT

Solo permitido si:

- $S0/S1 = 0$;

- deuda no afecta datos o Golden Path;
- workaround es razonable;
- owner y plan existen;
- el gate permite aceptación;
- queda registrada en known issues.

34. Builds reproducibles

34.1. Objetivo realista

Unity puede producir diferencias binarias por timestamps, compresión, entorno o toolchain. El objetivo inicial es **reproducibilidad auditable**, no prometer hashes idénticos sin demostrarlo.

Una build es auditable si puede reconstruirse con:

- mismo SHA;
- misma Unity;
- mismos paquetes/lock;
- mismos profiles/settings;
- mismos assets;
- mismos argumentos;
- mismo backend;
- registro del entorno.

34.2. Determinismo

La compilación C# determinista está activada. Esto no garantiza por sí solo determinismo del paquete completo.

34.3. Clean build

Antes de hito:

- checkout limpio;

- no reutilizar carpeta de salida;
- limpiar solo caches aprobadas cuando sea necesario;
- no borrar `Library` como ritual si el proyecto funciona;
- si se regenera `Library`, registrar el evento;
- esperar importación completa;
- ejecutar tests después.

34.4. Comparación

Para investigar divergencias:

- comparar manifest;
- comparar tamaños;
- comparar logs;
- comparar lista de escenas;
- comparar paquetes;
- comparar settings;
- comparar hashes por archivo si se necesita.

35. Automatización por línea de comandos

Esta sección es diseño objetivo. Debe validarse con el editor instalado antes de convertirla en pipeline oficial.

35.1. Ejecución de tests

Ejemplo conceptual:

```
& $UnityExe `  
-batchmode `  
-nographics `  
-quit `  
-projectPath $ProjectPath `  
-runTests `  
-testPlatform EditMode `
```

```
-testResults $EditModeXml `
-logFile $EditModeLog
```

Repetir para PlayMode. El script debe devolver error si faltan resultados o hay fallos.

35.2. Build

```
& $UnityExe `
  -batchmode `
  -quit `
  -projectPath $ProjectPath `
  -executeMethod VRMGames.CartridgeAndCloud.Editor.Build.BuildCommand.BuildWindows `
  -buildVersion 0.0.17 `
  -buildId S16-WIN-DEV-004 `
  -outputPath $Output `
  -logFile $BuildLog
```

La clase del ejemplo no existe actualmente y no debe invocarse hasta implementarla.

35.3. Argumentos

La herramienta futura debe validar:

- versión;
- Build ID;
- output path;
- perfil/canal;
- SHA esperado;
- escenas;
- schema;
- limpieza de salida;
- overwrite explícito.

35.4. Códigos de salida

Código	Significado propuesto
0	éxito
10	argumentos inválidos
20	working copy/preflight inválido
30	tests fallidos
40	fallo de build
50	artefacto incompleto
60	empaquetado/hash fallido
70	publicación fallida

Unity puede devolver códigos propios; el wrapper los traduce y conserva el original.

36. Diseño de BuildCommand

36.1. Responsabilidades

Una futura herramienta Editor debe:

- resolver argumentos;
- leer versión;
- verificar SHA;
- seleccionar configuración;
- validar escenas;
- crear output único;
- invocar `BuildPipeline`;
- inspeccionar `BuildReport`;
- generar manifest;

- escribir resumen;
- devolver código no cero ante fallo.

36.2. No responsabilidades

No debe:

- cambiar gameplay;
- reparar assets automáticamente;
- modificar schema;
- hacer commit/tag sin paso explícito;
- subir a Steam por defecto;
- ocultar warnings;
- sobrescribir un candidato aprobado.

36.3. Pseudocódigo

```
public static int BuildWindows(BuildRequest request)
{
    PreflightResult preflight = preflightService.Validate(request);
    if (!preflight.IsSuccess)
        return ExitCodes.PreflightFailed;

    BuildPlayerOptions options = optionsFactory.Create(request);
    BuildReport report = BuildPipeline.BuildPlayer(options);

    if (report.summary.result != BuildResult.Succeeded)
        return ExitCodes.BuildFailed;

    manifestWriter.Write(request, report);
    return ExitCodes.Success;
}
```

Debe seguir [09_CSharp_Coding_Standards.md](#): dependencias explícitas, funciones pequeñas, resultado estructurado y código Editor aislado.

37. Integración continua futura

37.1. Pipeline recomendado

```
checkout SHA
→ validar Unity/packages
→ cache controlada
→ compilar
→ EditMode
→ PlayMode
→ validar catálogos/escenas
→ build según gate
→ manifest
→ archivar tests/logs
→ smoke automatizable limitado
→ paquete/checksum
→ aprobación manual
```

37.2. Seguridad

- licencia Unity mediante mecanismo seguro;
- secretos en almacén del CI;
- permisos mínimos;
- artefactos privados;
- no imprimir tokens;
- proteger ramas/tags de release;
- revisar acciones de terceros.

37.3. Caché

La caché acelera, pero no puede ocultar dependencia ausente. Ejecutar periódicamente un build sin caché y siempre que se investigue una divergencia.

37.4. Gate manual

La automatización no reemplaza:

- aceptación visual;
- Golden Path;
- revisión de `Player.log`;
- playtest;
- aprobación de release.

37.5. Retención CI

- logs/test XML: retención media;
- builds ordinarias: corta;
- candidatos: hasta superseded + margen;
- releases: permanente;
- símbolos: según política de soporte.

38. Archivo y retención

38.1. Clases

Clase	Retención
intento fallido local	hasta resolver/registrar
build ordinaria	hasta cerrar sprint o ser superseded
build QA	durante campaña + periodo de diagnóstico
candidato de hito	permanente o largo plazo
demo pública	permanente
Alpha/Beta externa	largo plazo
RC	permanente
Release/Hotfix	permanente

38.2. Qué archivar

- ZIP o artefacto de tienda;
- checksum;
- manifest;
- build record;
- tests;
- logs;
- known issues;
- release notes;
- source SHA/tag;
- símbolos privados cuando existan;
- third-party notices aplicables.

38.3. Ubicaciones

No depender de una sola máquina. Debe existir al menos:

- copia primaria controlada;
- copia de backup;
- repositorio para registros pequeños;
- almacenamiento de artefactos para binarios.

No subir builds grandes a Git ordinario salvo decisión específica.

39. Rollback

39.1. Condiciones

Debe poderse identificar:

- última build aprobada;

- save compatibility;
- canal afectado;
- assets y manifest;
- instrucciones de reversión;
- riesgos de downgrade.

39.2. Rollback de código

No equivale siempre a `git revert`. Puede requerir:

- restaurar catálogo;
- revertir schema o mantener lector nuevo;
- preservar migración ya ejecutada;
- corregir Steam branch;
- informar a testers.

39.3. Saves

Nunca recomendar downgrade sobre un save migrado sin prueba. Opciones:

- backup previo;
- copia del slot;
- migración reversible probada;
- bloqueo de downgrade;
- hotfix forward.

39.4. Registro

Toda retirada/rollback documenta:

- motivo;
- build retirada;
- build restaurada;
- hora;
- impacto;

- compatibilidad;
- comunicación;
- seguimiento.

40. Hotfix

40.1. Cuándo

- crash frecuente;
- corrupción/pérdida;
- bloqueo del Golden Path;
- fallo de arranque;
- vulnerabilidad;
- asset/licencia indebida;
- regresión crítica tras distribución.

40.2. Flujo

```
reproducir en build afectada
→ crear rama hotfix
→ test de reproducción
→ corrección mínima
→ tests dirigidos
→ regresión vecina
→ suite/gate requerido
→ build nueva
→ upgrade/save test
→ Player.log
→ paquete/hash
→ release notes
→ publicación/rollback
```

40.3. Versionado

Un hotfix público incrementa PATCH. Un hotfix de una build interna puede conservar versión y subir Build iteration, siempre que no se confundan artefactos ya entregados.

40.4. Scope

No mezclar nuevas funciones con hotfix. Cualquier refactor amplio se difiere salvo que sea la única corrección segura demostrada.

41. Steam y distribución futura

41.1. Estado

Steam Publishing está [Planned](#) / [Deferred](#). Sprint 16 y Sprint 17 solo garantizan compatibilidad Windows y estructura de build; no implementan Steamworks.

41.2. Requisitos previos

- H6 aprobado;
- identidad visual estable;
- licencias y créditos cerrados;
- rutas de save verificadas;
- opciones/localización adecuadas;
- build sin crashes;
- store assets;
- política de privacidad si aplica;
- soporte y rollback.

41.3. App, depots y branches

Objetivo conceptual:

- una App principal;
- depot Windows;
- depot de demo separado cuando proceda;
- branch `default` para release;
- branches privadas para QA, playtest, beta y candidatos;
- contraseñas y permisos fuera del repositorio.

Los IDs reales no se inventan en documentación antes de crearse.

41.4. Steam BuildID

Después de una subida, registrar:

- AppID;
- Depot IDs;
- Manifest IDs;
- Steam BuildID;
- branch;
- SHA del paquete fuente;
- fecha;
- aprobación.

41.5. SteamPipe

El pipeline futuro debe separar:

- generación local;
- staging;
- validación;
- subida;
- asignación de branch;
- promoción.

Una subida exitosa no equivale a publicación pública.

41.6. Steam Cloud

No activar hasta definir:

- archivos y patrones;
 - tamaño;
 - conflictos;
 - multi-machine;
 - recuperación;
 - compatibilidad con backups locales;
 - borrado;
 - pruebas offline.
-

42. Demo

42.1. Build separada

La demo no debe ser una Release completa limitada solo por comunicación. Debe definir:

- alcance;
- contenido;
- saves;
- transferencia a juego completo;
- límites de tiempo/progresión;
- branding;
- App/depot si Steam;
- soporte.

42.2. Versionado

Ejemplo:

0.3.1-demo.1

El número final se decide en gate. Debe diferenciarse del juego completo en manifest y UI.

42.3. Save transfer

Si se permite continuar en la versión completa:

- probar upgrade;
- evitar IDs específicos de demo incompatibles;
- documentar contenido que se descarta;
- conservar backup;
- impedir que la demo abra saves futuros.

43. Licencias, créditos y seguridad

43.1. Licencias

Antes de distribución:

- revisar assets;
- audio;
- fuentes;
- plugins;
- paquetes;
- código externo;
- marcas y carátulas;
- `THIRD_PARTY_NOTICES`;
- créditos.

No usar marcas o carátulas reales sin autorización.

43.2. Secretos

No incluir en Player, ZIP, logs o repositorio:

- credenciales Steam;
- claves privadas;
- tokens API;
- contraseñas de branches;
- rutas con secretos;
- certificados privados.

43.3. Escaneo

Antes de release pública, considerar:

- antivirus del artefacto;
- inventario de dependencias;
- revisión de archivos inesperados;
- búsqueda de secretos;
- validación de firma cuando exista.

44. Checklist Sprint 16

44.1. Entrada

- versión `0.0.17` confirmada;
- working copy identificada;
- `StoreInitial` autorada;
- contexto conectado;
- catálogo runtime válido;
- fallbacks controlados.

44.2. Perfil

- no cambiar Scene List antes de conectar runtime;
- después de validar, reemplazar destino `Store` por `StoreInitial`;
- mantener `Bootstrap` índice 0;
- conservar `TestLab` solo en Development;
- registrar diff del Build Profile.

44.3. QA

- EditMode completo;
- PlayMode completo;
- aprobación visual;
- clic UI no mueve jugador;
- no hay doble shell;
- Golden Path post-integración;
- build Windows x64 externa;
- `Player.log` revisado;
- $S0/S1 = 0$.

44.4. Cierre

- Build ID;
 - SHA;
 - escenas;
 - manifest;
 - evidencia;
 - closure record;
 - handoff;
 - no abrir Sprint 17 si falta cualquiera de los gates obligatorios.
-

45. Checklist Sprint 17 y H6

45.1. Sprint 17

- Sprint 16 cerrado;
- versión objetivo congelada;
- balance registrado;
- profiling en build;
- persistencia/recovery;
- accesibilidad;
- localización ES/EN;
- defectos clasificados;
- suite completa;
- Golden Path;
- campaña de siete días;
- `Player.log`;
- build Windows x64 identificada.

45.2. Candidato H6

- checkout limpio;
- perfil Candidate;
- `TestLab` excluido;
- StoreInitial como escena de juego;
- Development flags revisados;
- manifest;
- tests;
- rendimiento y hardware;
- saves y recovery;
- ZIP;
- SHA-256;
- known issues;

- vídeo/capturas;
- decisión PASS/PASS WITH ACCEPTED DEBT/FAIL.

45.3. Tras PASS

- tag de hito si se publica;
- release interna o artefacto archivado;
- baseline documental;
- retrospectiva;
- no reutilizar el mismo nombre para una build posterior.

46. Alpha, Beta, RC y Release

46.1. Alpha

- todas las capacidades comprometidas existen;
- saves/migraciones funcionan;
- build reproducible/auditable;
- contenido puede estar incompleto;
- no se añaden sistemas mayores sin cambio formal;
- telemetría/crash reporting se decide antes de externa.

46.2. Beta

- feature complete;
- contenido casi completo;
- rendimiento y compatibilidad;
- accesibilidad/localización;
- Steam y distribución probados;
- actualización desde Alpha;

- known issues controlados.

46.3. Release Candidate

- no S0/S1;
- S2 aceptados;
- regresión completa;
- saves de versiones soportadas;
- instalación limpia y actualización;
- firma/Steam;
- legal/créditos;
- soporte;
- rollback;
- hash y símbolos.

46.4. Release

- RC sin cambios de contenido; cualquier cambio exige nueva RC;
- store build aprobada;
- branch correcta;
- release notes;
- monitorización;
- backups;
- canal de hotfix;
- artefactos archivados.

47. Definition of Done de una build

Una build se considera terminada cuando:

1. tiene Build ID único;

2. versión y SHA están registrados;
3. Unity y paquetes están identificados;
4. perfil y escenas son correctos;
5. schema está declarado;
6. preflight pasa;
7. tests requeridos pasan;
8. Unity completa la build;
9. artefactos mínimos existen;
10. el Player arranca fuera del Editor;
11. smoke/Golden Path requerido pasa;
12. Quit funciona;
13. `Player.log` se revisa cuando el gate lo exige;
14. defectos están clasificados;
15. no hay S0/S1 para candidato;
16. evidencia está archivada;
17. manifest existe para build retenida;
18. paquete y hash existen cuando proceden;
19. known issues están congelados;
20. existe decisión formal.

Una build local que “abre” puede ser útil, pero no está `DONE` como artefacto de hito.

48. Patrones prohibidos

- sobrescribir una build aprobada;
- llamar `Final` a un candidato;
- modificar `bundleVersion` después de empaquetar;
- crear tag antes de QA;
- producir build desde working copy desconocida;
- incluir `TestLab` en distribución sin justificación;

- arrancar desde `StoreInitial` saltando Bootstrap;
- ignorar un `Player.log` con excepciones recurrentes;
- ocultar fallos borrando logs;
- invalidar saves sin aviso;
- cambiar Application Identifier informalmente;
- cambiar backend en el último momento;
- excluir tests para obtener verde;
- distribuir ZIP sin saber qué SHA contiene;
- subir secretos a Steam scripts o repositorio;
- reutilizar Build ID;
- mezclar artefactos de dos builds;
- publicar known issues desactualizados;
- considerar `Build Succeeded` equivalente a aceptación;
- inventar tamaño, hash o ruta no registrados.

49. Criterios de aceptación de la política

ID	Criterio	Evidencia mínima
BUILD-STD-001	Toda build retenida tiene Build ID único.	Registro, manifest, test o artefacto aplicable
BUILD-STD-002	Toda build retenida registra versión de aplicación y SHA completo.	Registro, manifest, test o artefacto aplicable
BUILD-STD-003	Bootstrap ocupa el índice 0 en toda build jugable.	Registro, manifest, test o artefacto aplicable
BUILD-STD-004	Una build distribuida excluye TestLab salvo excepción aprobada.	Registro, manifest, test o artefacto aplicable
BUILD-STD-005	StoreInitial sustituye a Store solo después de integración validada.	Registro, manifest, test o artefacto aplicable
BUILD-STD-006	No se aprueba candidato con S0 o S1 abierto.	Registro, manifest, test o artefacto aplicable
BUILD-STD-007	Todo candidato de hito incluye Player.log revisado.	Registro, manifest, test o artefacto aplicable
BUILD-STD-008	Todo candidato de hito incluye paquete y SHA-256.	Registro, manifest, test o artefacto aplicable
BUILD-STD-009	Todo cambio de schema incluye compatibilidad, migración o rechazo explícito.	Registro, manifest, test o artefacto aplicable
BUILD-STD-010	Una carga de save se valida por equivalencia observable.	Registro, manifest, test o artefacto aplicable

ID	Criterio	Evidencia mínima
BUILD-STD-011	Los IDs de contenido persistido no se reutilizan.	Registro, manifest, test o artefacto aplicable
BUILD-STD-012	La versión documental se distingue de la versión de aplicación.	Registro, manifest, test o artefacto aplicable
BUILD-STD-013	Un tag representa un gate aprobado, no un intento.	Registro, manifest, test o artefacto aplicable
BUILD-STD-014	Una release publicada no se sobrescribe ni mueve.	Registro, manifest, test o artefacto aplicable
BUILD-STD-015	Los datos no registrados se declaran como no registrados.	Registro, manifest, test o artefacto aplicable
BUILD-STD-016	El artefacto se ejecuta fuera del Editor.	Registro, manifest, test o artefacto aplicable
BUILD-STD-017	La lista de escenas se verifica antes de construir.	Registro, manifest, test o artefacto aplicable
BUILD-STD-018	La build usa la versión exacta de Unity aprobada.	Registro, manifest, test o artefacto aplicable
BUILD-STD-019	Los paquetes se resuelven desde manifest y lock versionados.	Registro, manifest, test o artefacto aplicable
BUILD-STD-020	Una build de profiling se identifica y no sustituye una medición final.	Registro, manifest, test o artefacto aplicable
BUILD-STD-021	Los secretos no entran en Player, logs, scripts o repositorio.	Registro, manifest, test o artefacto aplicable
BUILD-STD-022	El cambio de backend requiere ADR y regresión completa.	Registro, manifest, test o artefacto aplicable
BUILD-STD-023	El cambio de Application Identifier requiere plan de saves.	Registro, manifest, test o artefacto aplicable
BUILD-STD-024	Un hotfix incluye reproducción y regresión dirigida.	Registro, manifest, test o artefacto aplicable
BUILD-STD-025	Un rollback evalúa saves migrados antes de ejecutarse.	Registro, manifest, test o artefacto aplicable
BUILD-STD-026	Todo gate H6 incluye Golden Path y campaña persistente.	Registro, manifest, test o artefacto aplicable
BUILD-STD-027	El checksum se calcula sobre el paquete definitivo.	Registro, manifest, test o artefacto aplicable
BUILD-STD-028	Los known issues se congelan por candidato.	Registro, manifest, test o artefacto aplicable
BUILD-STD-029	Un CI futuro falla ante tests o build fallidos.	Registro, manifest, test o artefacto aplicable
BUILD-STD-030	La automatización no reemplaza la aprobación visual/manual.	Registro, manifest, test o artefacto aplicable

50. Matriz de decisión por tipo de cambio

Cambio	¿Nueva build?	¿Nueva versión app?	¿Schema?	¿ADR?	Regresión
documentación únicamente	no obligatoria	no	no	normalmente no	revisión documental
corrección C# sin distribución	sí para validar	patch/sprint según gate	si cambia datos	según alcance	dirigida + vecinos

Cambio	¿Nueva build?	¿Nueva versión app?	¿Schema?	¿ADR?	Regresión
asset visual	sí	según sprint	no salvo referencias persistentes	no normalmente	escena + Golden Path
cambio de catálogo	sí	sí si se integra	quizá content/schema	según riesgo	datos + save/load
cambio de snapshot	sí	sí	sí	normalmente	fixtures + migración
cambio de escena de build	sí	no necesariamente	no	si arquitectura	smoke completo
Mono → IL2CPP	sí	probablemente	no	sí	completa + rendimiento
Application Identifier	sí	sí	no	sí	instalación/save migration
hotfix público	sí	patch	si aplica	no siempre	reproducción + completa mínima
paquete Unity	sí	según gate	quizá	sí si material	completa
resolución/UI	sí	según sprint	no	no	resoluciones + UX
Steam integration	sí	minor/patch según fase	quizá	sí	offline/online/store

51. Plantilla de preflight

```
# Build Preflight – <Build ID>

## Source
- Branch:
- Commit SHA:
- Working copy clean:

## Versions
- Application:
- Unity:
- Packages:
- Schema:
- Content:

## Profile
- Profile:
- Platform:
- Architecture:
- Backend:
- Flags:
- Scenes:

## Validation
- Compile:
- EditMode:
- PlayMode:
- Catalog validation:
- Save fixtures:
```

```
- Open S0/S1:

## Decision
PASS / BLOCKED / FAIL
```

52. Plantilla de manifest JSON

```
{
  "buildId": "H6-WIN-CAND-001",
  "applicationVersion": "0.3.0",
  "buildIteration": 1,
  "commitSha": "0000000000000000000000000000000000000000",
  "builtAtUtc": "2026-07-01T00:00:00Z",
  "unityVersion": "6000.3.18f1",
  "profile": "Windows_H6_Candidate",
  "platform": "Windows",
  "architecture": "x86_64",
  "backend": "Mono",
  "schemaVersion": 2,
  "scenes": ["Bootstrap", "MainMenu", "StoreInitial"],
  "tests": {
    "editMode": {"passed": 0, "failed": 0},
    "playMode": {"passed": 0, "failed": 0}
  },
  "artifactSha256": ""
}
```

53. Plantilla de release record

```
# Release Record – <Version / Channel>

## Approval
- Gate:
- Decision:
- Approved by:
- Date:

## Artifact
- Build ID:
- File:
- SHA-256:
- Size:
```

```
- Storage:

## Source
- Tag:
- Commit:
- Baseline documentation:

## Compatibility
- Save schema:
- Supported previous versions:
- Upgrade path:
- Downgrade policy:

## Distribution
- Channel/branch:
- Steam BuildID, if any:
- Publication time:

## Known issues

## Rollback
```

54. Historial de builds verificado

Hito	Versión	Tipo	Resultado	Evidencia retenida
Sprint 0 Build001	0.0.1	Development	PASS con observaciones	ZIP, SHA-256, log y capturas
Sprint 0 Build002	0.0.1	Final Development	PASS	ZIP, tamaño, SHA-256 y log
Sprint 3	0.0.4	Development	PASS	record; sin ZIP/hash
Sprint 4	0.0.5	Development	PASS	record; sin ZIP/hash
Sprint 5	0.0.6	Development	PASS	record; sin ZIP/hash
Sprint 7	0.0.8	Development	PASS	warning Unity Services no bloqueante
Sprint 8	0.0.9	Development	PASS	record operativo
Sprint 9	0.0.10	Development	PASS	ruta/tamaño/hash no registrados
Sprint 10	0.0.11	Development	PASS	ruta/tamaño/hash/log no exportados
Sprint 11	0.0.12	Development	PASS	record operativo
Sprint 12	0.0.13	Development	PASS	record operativo
Sprint 13	0.0.14	Development	PASS	record operativo
Sprint 14	0.0.15	Development	PASS	save/recovery validados
Sprint 15	0.0.16	Development	PASS	UI, slots y autosave validados

Hito	Versión	Tipo	Resultado	Evidencia retenida
Sprint 16 preintegración	0.0.17	Development	PASS documentado	requiere nueva build post-integración

54.1. Lecciones

- un build puede ser válido sin retener ZIP en cada sprint;
- los campos ausentes no deben inventarse;
- los hashes tienen más valor en hitos;
- PlayMode assemblies deben usar `UNITY_INCLUDE_TESTS` para no entrar al Player;
- los warnings de Unity Services son no bloqueantes mientras no exista dependencia;
- la transición a StoreInitial exige nueva evidencia, no reutilizar el PASS anterior.

55. Deuda y evolución recomendada

Prioridad inmediata

1. completar `StoreInitial`;
2. actualizar perfil después de conectar runtime;
3. generar build post-integración;
4. revisar `Player.log`;
5. crear registro completo;
6. abrir Sprint 17 solo tras cerrar Sprint 16.

Antes de H6

- perfil H6 Candidate;
- manifest de build;
- script mínimo de validación de escenas/versión;
- paquete y SHA-256;
- campaña de siete días;
- hardware y rendimiento;

- known issues congelados.

Antes de demo/Alpha

- comando de build reproducible;
- CI o ejecución scripted;
- content manifest;
- política de símbolos;
- crash reporting evaluado;
- localización y licencias;
- perfil sin Development;
- pruebas de actualización.

Antes de Release

- backend final;
- firma;
- SteamPipe;
- branches/depots;
- rollback;
- soporte/hotfix;
- seguridad y secretos;
- archivo permanente.

56. Taxonomía de fallos de build

Los fallos deben clasificarse antes de corregirse. Una clasificación correcta evita aplicar soluciones destructivas, como borrar [Library](#), cambiar paquetes o desactivar tests sin conocer la causa.

56.1. Fallo de compilación C#

Síntomas:

- `CSxxxx` en consola;
- assembly que no compila;
- tipo o namespace ausente;
- referencia circular;
- código de test incluido en Player.

Respuesta:

1. conservar el primer error y su contexto;
2. identificar assembly y cambio que lo introdujo;
3. revisar `.asmdef`, define constraints y referencias;
4. corregir el error raíz antes de los secundarios;
5. ejecutar compilación y tests dirigidos;
6. no cambiar paquetes para ocultar un error de código.

El incidente histórico de S0.9 mostró que un assembly PlayMode sin `UNITY_INCLUDE_TESTS` puede entrar en el Player y hacer fallar la build. La corrección correcta fue ajustar el assembly, no eliminar las pruebas.

56.2. Fallo de importación o asset

Síntomas:

- GUID faltante;
- material rosa;
- referencia `Missing`;
- FBX o textura que no importa;
- prefab inválido;
- catalog validation fallida.

Respuesta:

- comprobar `.meta`;
- revisar origen del asset;

- validar que no se movió fuera de Unity;
- reimportar solo el asset o carpeta afectada;
- comparar contra el commit;
- no regenerar GUID manualmente;
- verificar licencias y tamaño.

56.3. Fallo de Scene List

Síntomas:

- build arranca en escena equivocada;
- `Scene couldn't be loaded;`
- MainMenu no encuentra `StoreInitial;`
- escena presente pero deshabilitada;
- TestLab aparece en distribución.

Respuesta:

1. inspeccionar Build Profile y lista global;
2. comparar ruta y GUID;
3. confirmar Bootstrap en índice 0;
4. revisar la configuración runtime que contiene el nombre de escena;
5. ejecutar smoke externo;
6. actualizar QA y ADR si cambió el contrato.

56.4. Fallo de Player distinto del Editor

Posibles causas:

- stripping;
- orden de inicialización;
- paths;
- case sensitivity de nombres;
- código protegido por `UNITY_EDITOR;`
- assets no incluidos;

- diferencias de timing;
- serialización;
- plugins;
- permisos de archivo.

Respuesta:

- conservar `Player.log`;
- reproducir con el mismo artefacto;
- activar una build diagnóstica separada si hace falta;
- no modificar el candidato original;
- escribir test que capture el defecto cuando sea viable;
- comprobar que la corrección funciona en Player, no solo en Editor.

56.5. Fallo de empaquetado o hash

Síntomas:

- ZIP incompleto;
- doble carpeta raíz;
- hash no coincide;
- ejecutable bloqueado durante compresión;
- archivo de otra build incluido.

Respuesta:

- cerrar procesos;
- borrar solo el paquete fallido, no la carpeta fuente validada;
- volver a empaquetar desde staging limpio;
- probar extracción;
- recalcular hash;
- incrementar Build iteration si el artefacto ya se comunicó.

56.6. Fallo de almacenamiento

- disco lleno;

- ruta demasiado larga;
- permisos insuficientes;
- antivirus bloquea archivo;
- backup no disponible.

El preflight debe medir espacio y verificar escritura. No se debe aprobar un artefacto cuyo único original esté en una ruta temporal.

57. Entorno y hardware del build

57.1. Registro del entorno

Para builds de hito conservar:

Campo	Ejemplo
OS	Windows, edición y build
CPU	modelo
RAM	capacidad
GPU/driver	si afecta shader/perfil
Disco	tipo y espacio disponible
Unity	versión/revisión
Hub	opcional
Git	versión
Locale	idioma/región
Zona horaria	UTC/local
Antivirus	observación si interviene

No se pretende que cada build ordinaria tenga un inventario exhaustivo. El objetivo es poder explicar diferencias de rendimiento, paths, compresión o importación.

57.2. Hora y fecha

- manifest usa UTC;
- nombres pueden usar fecha local acordada;
- registros indican zona horaria;
- no confiar solo en timestamp del sistema de archivos;
- sincronizar reloj antes de firma o publicación.

57.3. Máquina de build

Antes de release conviene una máquina o runner controlado:

- software mínimo;
- acceso restringido;
- toolchain fijado;
- sin plugins personales;
- almacenamiento cifrado de secretos;
- logs preservados;
- capacidad de reconstrucción.

Mientras se use la máquina de desarrollo, registrar cualquier diferencia material y evitar builds con herramientas experimentales activas.

58. Procedimiento de bump de versión

58.1. Bump ordinario de sprint

1. revisar el charter;
2. decidir versión objetivo;
3. comprobar última versión publicada;
4. actualizar `PlayerSettings.bundleVersion`;
5. actualizar registro de sprint;

6. ejecutar test que lea la versión, si existe;
7. verificarla en build externa;
8. no cambiarla durante la aceptación salvo que se invalide la build.

58.2. Bump por hotfix

- partir de la versión afectada;
- incrementar PATCH;
- conservar compatibilidad de schema o documentar migración;
- actualizar release notes;
- no incorporar features de la rama principal sin necesidad;
- probar actualización desde el artefacto anterior.

58.3. Bump de schema

La secuencia correcta es:

```
definir nueva forma
→ implementar migrador
→ conservar reader anterior
→ crear fixtures
→ tests de migración
→ incrementar constante de schema
→ build
→ validación de saves
```

Incrementar primero la constante y “arreglar después” crea una ventana en la que el programa puede escribir datos sin lector fiable.

58.4. Bump documental

- actualizar front matter;
- conservar fecha;
- añadir changelog;
- generar hash;
- no cambiar la versión de aplicación si no cambió el Player.

58.5. Bump de contenido

Cuando exista `contentManifestVersion`:

- calcularlo a partir de catálogo aprobado o incrementarlo mediante procedimiento;
- vincularlo al manifest de build;
- probar IDs faltantes;
- documentar cambios de balance significativos;
- no sustituir schema por content version cuando cambia estado persistente.

59. Matriz de artefactos por gate

Artefacto	Sprint ordinario	Sprint 16	Sprint 17	H6	Demo/Release
Build record	sí	sí	sí	sí	sí
SHA de commit	sí	sí	sí	sí	sí
Test summary	sí	completo	completo	completo	completo
Player.log	condicional	obligatorio	obligatorio	obligatorio	obligatorio
Golden Path	según alcance	obligatorio	obligatorio	obligatorio	obligatorio
ZIP	opcional	recomendado cierre	sí	obligatorio	obligatorio
SHA-256 ZIP	opcional	si se retiene	sí	obligatorio	obligatorio
Manifest	objetivo	recomendado	sí	obligatorio	obligatorio
Known issues snapshot	básico	sí	sí	sí	sí
Hardware/performance	no siempre	smoke	obligatorio	obligatorio	obligatorio
Capturas/vídeo	opcional	aprobación visual	recomendado	obligatorio	marketing/QA
Tag	no	no por defecto	decisión de hito	si se publica	sí
Release record	no	no por defecto	interno	sí	sí
Third-party notices	no	revisión	revisión	revisión	obligatorio
Rollback plan	no	fallback escena	sí	sí	obligatorio

59.1. Regla de proporcionalidad

La evidencia aumenta con el coste de distribución y recuperación. No se exige ceremonia de release a cada build local, pero tampoco se permite usar una build local como candidato sin completar sus artefactos.

60. Escenarios de migración y actualización

60.1. Nueva instalación

Validar:

- arranque sin carpeta previa;
- creación de directorio de datos;
- MainMenu y slot vacío;
- opciones por defecto;
- primer save;
- salida limpia.

60.2. Actualización compatible

- instalar o sustituir Player conservando datos;
- iniciar;
- detectar saves existentes;
- cargar cada slot soportado;
- guardar con schema vigente;
- reiniciar;
- comprobar equivalencia.

60.3. Actualización con migración

- backup previo;
- cargar fixture antiguo;
- migrar en memoria;
- validar;
- escribir nuevo primario;
- preservar backup anterior según política;

- registrar migración;
- impedir doble migración.

60.4. Save futuro

Una build antigua que recibe un schema futuro debe:

- identificar incompatibilidad;
- no sobrescribir;
- no “reparar” destructivamente;
- mantener backup;
- mostrar mensaje útil;
- permitir volver al menú/salir.

60.5. Cambio de Application Identifier

Si alguna vez es imprescindible:

- mapear ruta antigua y nueva;
- copiar, no mover inicialmente;
- validar checksums;
- manejar conflictos;
- ofrecer rollback;
- probar perfiles de usuario sin permisos elevados;
- documentar la transición.

60.6. Demo a juego completo

- resolver App/depots;
- decidir si comparte carpeta de save;
- marcar origen de la partida;
- migrar contenido limitado;
- evitar referencias a assets ausentes;
- validar logros/Steam Cloud si existen;

- comunicar qué progreso se conserva.

61. Revisión formal de un candidato

61.1. Reunión o acto de gate

Aunque el desarrollo sea individual, realizar una revisión separada:

1. leer objetivo;
2. identificar artefacto sin reconstruirlo;
3. confirmar SHA/hash;
4. revisar pruebas;
5. ejecutar o revisar Golden Path;
6. inspeccionar `Player.log`;
7. revisar defectos;
8. revisar saves y rendimiento;
9. decidir PASS, deuda o FAIL;
10. registrar siguiente acción.

61.2. Preguntas de rechazo

- ¿el artefacto puede no corresponder al SHA?;
- ¿hay un S0/S1?;
- ¿se perdió evidencia?;
- ¿el profile contiene escenas incorrectas?;
- ¿el save puede dañarse?;
- ¿el candidato fue modificado después del hash?;
- ¿la aprobación visual sigue pendiente?;
- ¿se utilizó una build de profiling como final?;
- ¿las licencias están sin resolver?;

Un “sí” material produce FAIL o bloqueo, no una observación decorativa.

61.3. Cierre

Después del PASS:

- bloquear el artefacto;
- copiar a archivo;
- publicar tag/release cuando proceda;
- actualizar baseline;
- marcar builds anteriores como superseded, no borrarlas si son hitos;
- comunicar canal y known issues.

62. Runbook de recuperación

62.1. La build candidata no arranca

- no reconstruir sobre la misma carpeta;
- conservar logs;
- probar en la máquina de build y otra máquina si existe;
- verificar dependencias y cuarentena antivirus;
- volver al último candidato aprobado;
- abrir defecto S1;
- producir nueva iteración después del fix.

62.2. El hash publicado no coincide

- detener distribución;
- comparar archivo y fuente;
- retirar checksum incorrecto;
- determinar si cambió el ZIP o el registro;

- no “corregir” el hash sin explicar el incidente;
- publicar artefacto/registro nuevo y dejar trazabilidad.

62.3. Se incluyó un secreto

- retirar artefacto;
- revocar/rotar secreto;
- limpiar historial cuando proceda;
- comprobar descargas y logs;
- abrir incidente de seguridad;
- producir build nueva;
- no confiar en borrar solo el archivo visible.

62.4. Corrupción de saves tras update

- retirar/promover rollback según seguridad;
- preservar saves afectados;
- no pedir al usuario que los sobrescriba;
- identificar schema/build;
- construir herramienta o hotfix de recuperación;
- probar con copias;
- comunicar workaround seguro.

62.5. Build profile dañado

- comparar con Git;
- restaurar asset y `.meta` juntos;
- no recrear a ciegas si cambia GUID;
- validar escenas;
- ejecutar smoke;
- registrar por qué se dañó.

62.6. Paquete o Unity cambió accidentalmente

- detener build;
- revisar `manifest`, lock y `ProjectVersion`;
- restaurar baseline;
- reimportar;
- ejecutar suite;
- no aceptar el cambio como “actualización automática”.

Anexo A. Rutas relevantes

```
ProjectSettings/ProjectVersion.txt
ProjectSettings/ProjectSettings.asset
ProjectSettings/EditorBuildSettings.asset
Assets/_Project/Settings/BuildProfiles/Windows_Development.asset
Packages/manifest.json
Packages/packages-lock.json
Documentation/10_Development_Records/Builds/
Documentation/10_Development_Records/Sprint_*/QA/*Build*Record.md
Documentacion/08_QA_Testing_Matrix.xlsx
```

Anexo B. Valores protegidos

Cambiar cualquiera requiere revisión formal y, en varios casos, ADR:

- Unity `6000.3.18f1`;
- Windows x64 como plataforma inicial;
- `Bootstrap` como entrada;
- `ApplicationRoot` único;
- Application Identifier;
- company/product name;
- backend;

- schema;
- ruta/ownership de saves;
- orden de escenas;
- política de tags;
- perfil Release;
- Steam IDs futuros;
- firma y certificados.

Anexo C. Definiciones

Término	Definición
Artefacto	archivo o conjunto producido por build
Baseline	conjunto aprobado de código, configuración, pruebas y documentos
Build	proceso y/o resultado compilado, según contexto
Build ID	identificador único de una iteración
Candidate	artefacto sometido a gate
Channel	destino de distribución
Checksum	hash de integridad
Clean build	build desde estado limpio/controlado
Depot	contenedor de archivos en Steam
Manifest	metadata de identidad y configuración
RC	Release Candidate
Schema	versión de estructura persistente
SHA	identificador de commit o hash de archivo, según campo
Smoke	recorrido mínimo de viabilidad
Superseded	reemplazado por artefacto posterior

Anexo D. Ruta final

Documentacion/
└─ 11_Build_y_Versioning_Guide.md

Estado del documento: fuente vigente para builds y versionado. Debe actualizarse al cerrar Sprint 16, congelar la versión de Sprint 17, producir el candidato H6 o adoptar automatización, IL2CPP, firma o Steam.

Anexo E. Trazabilidad de fuentes

Fuente	Bytes	SHA-256
00_Enfoque_y_Alcance.md	127401	63dd6f0f1b9c2c80be2aec55718d18d9d031ce4acb14f677769d6e69ceaad21f
01_Game_Design_Document.md	132822	a9cd9e3203abc5269c25c59dc7f626b0771df4ec2d65e781109ff146bceb2177
02_Vertical_Slice_Specification.md	64531	75893f130116e88a08b8da5590dd81876a234581081eafaa8d08374c1a60513f
03_Technical_Design_Document.md	101994	66264703c5ff4959d03093690e9845a98ec0e5025e685b9a639ac8cbb616cd12
04_Modelo_de_Datos.md	102948	d851a72b55742c2c704cc7c002929bc5894e7142511c6af843440bb193fb19d4
05_UX_Flow.md	56805	e08da5ff67fb073a8b950babe325ae58ddb2e121513dc2f553acd4f2c14b867e
06_Production_Roadmap_y_Sprint_Plan.md	61564	d00b156c0ad1c66069278119c55f76c738a577a3e2835f770208d1a5f2faadff
07_QA_Testing_Plan.md	79435	bc9c05a3737e345546ef16e5390e034295ef35ad66c31647852aec81b4e7affe
08_QA_Testing_Matrix.xlsx	185032	233e57f01d2468fe13c136c8fcd3ee75b39bd3de6530349626ea98d4ff254214
09_CSharp_Coding_Standards.md	95321	3b734d6cd49f31c09c10bb4941625e143f6c634e3fb50c157f0720e73b1c2a68
10_Unity_Project_Setup_Guide.md	78408	31430bb0544edd9577323aa0009d6ca3df8b9fbcf6c8a150fc33df69b9f963a4
EditorBuildSettings.asset	722	220a1f0e6a56e964e25b64481b10049e3e4fad883000f3e67b7000596fe7bd32
EditorSettings.asset	1585	32d6a9be4c9aa9795a96bd951820a2d9be1bfefab82ed05a5db111af0863938f
ProjectSettings.asset	25630	19e2a914806164939a5cae16291c9b6f29f0a8b1b8a9f50d8fd74ea65c03178c
ProjectVersion.txt	85	d03fca18d2e511a37f7a81f8709404ebb2a571d3e0e5c854038c91a943ae15da

Fuente	Bytes	SHA-256
Windows_Development.asset	1946	63e658288030c33844a84c37fc636fa908e9d8fbe44d2180aefcf03a0e5c93fa
adr0005.md	1662	e83bbf7ee82448366ccdd997c18b11a620016ba4ea70b4576f8b159930db8880
adr0007.md	2251	e62888a6608c248e56868b73025bd7044c455528178984f410f53fa18e942641
adr0010.md	2861	ad328a9eabc2302b0688591df010650b693d9423f0f485eb1000e24d50475184
changelog_34.md	1071	3a7a302140f163b86c0e5328b9af5b669ad79ce0791537f6c59a0f735e0c7414
changelog_45.md	624	8d6ba200548c298501c95844d723d79a3337a66cff1ae15b5b9937981325884d9
changelog_56.md	477	04ef84bbe6d5aad3bec5cce41fd0bcefe9d23ea8f7263febc5bd97bae97aa4a
guide_v03.md	3678	fcc0988f428b45303c4cc2f5a681cbcd9970475667bb6a37fa97a36529dc3705
guide_v04.md	4723	6ac0c7072b5ac4b1844d82daffe1ec5f241dba369145e5c62152e290b8309acd
guide_v05.md	2384	3df0fc5321a57c528f6386bcd922c2e930ba194736af94f064262c371bb63be6
guide_v06.md	2462	8cd0f62e233cfbad2bed3a07122696e2ea81b4fd5280987cffb9a8a2a1e98d43
legal_v05.md	1951	610fc2ca1dd8ea1859fb779039e5de841610646d1e5601d316b2fe34410c3c81
manifest.json	2828	50137b7babf4c53742103d5f14c86fde2090a984c030292e2e4aaef4ce03dae6
packages-lock.json	41858	6af42e872600380cea023aed62c849bed1e4b204a9dfc9e88da8184b06b6bbe2
s0_10.md	1484	2bdddb6ad5bff23d564e9d179c0b31b5baa4dcf00624539571485725734231a2
s0_9.md	3786	740b19ff9c450ae57b34c0cf73c850db19365dacc250c4ebfd9c0e581e06a3b8
s10.md	535	adea9b7fcdab6e319f0ded3171260cc0200e2ae3ab424916a8068a7175872369
s11.md	494	6e5b3ee3b6b9a1a6d2f6c90fc07df567cfdd81126f6e2c06aa4c3307db206f24
s12.md	431	e3531d9bc28665f33ed38c176188246308d38bd480be42c1758f848e433f69b4
s13.md	493	887232c7346268536f197fc6069ad7f518735c2f51c302f0d27e6c2625cd005b
s14.md	490	758b82a0301e5f3de699b76365cf74c577377bcb6d88333c4b65a26e6065af33
s15.md	359	cb74c7605f9918b31b4f49ebbe0f63004b03886d5468bd258a3eb6f83ea9b61a
s3.md	1303	bdbcbe899c627a7abf1f302f7db42ce29de1f0809c8373a454e3bc070984ab7d
s4.md	1164	6f662c5fe7cce7a3080584798ccf51c720d4ade1dc50b1be2f8c4c107da529d
s5.md	1347	83700192228af99fb4487bdf4cdd1332d13a194be7cd5fb197278013952f8fe1
s7.md	1288	fa7507ba60b575ad8ec8b6a5e5dc6fa9531436bf0f33822e9965edef6b15bd6d
s8.md	1531	975e35974c3ac193dedc354ad39f652dccd31356a4d69ec241dddcfdd84f29c1

Fuente	Bytes	SHA-256
s9.md	773	743edb2e5421984273fb169aab3fbb16c48d6b0ac2e177c5e83ba01b51099304
steam_v05.md	1866	f7840ee791e07430dd750500421dc6e262d21d5119e634634919247ff13cf8d0

Fin del documento.

Fuente: 11_Build_y_Versioning_Guide.md · SHA-256: 38a16a0f97505c0b405509dd3c7e3d1b50e77b89fbe3f4cb1931d3e04f602215

Diseño PDF: paleta VRM Games definida en la documentación de Cartridge & Cloud.